

Caderno de Prática - SQL Intermediário (Módulo 1, Semana 10)

Sep 28, 2026 · @Alvaro P

Como usar este caderno

São 9 atividades, 3 por dia de aula, todas sobre a mesma base de dados: você acompanha a demonstração ao vivo e depois resolve uma tarefa sozinho.

Antes do Dia 1: carregue a base

1. Salve o arquivo `base_pratica_postgres.sql`, entregue pelo professor. É necessário **PostgreSQL 13 ou superior**.
2. **Pelo psql** (linha de comando): `psql -U seu_usuario -f base_pratica_postgres.sql`. O script usa o comando `\c` para trocar de banco no meio do arquivo, e isso só funciona no psql.
3. **Pelo pgAdmin**: abra uma conexão em qualquer banco (ex.: `postgres`) e rode só o trecho inicial do script, até o `CREATE DATABASE`. Depois crie, na árvore de conexões, uma nova conexão apontando para o banco `pratica_sql` e rode o restante do arquivo nessa conexão. O pgAdmin não troca de banco no meio de uma execução.
4. O script apaga e recria o banco `pratica_sql`, então pode ser executado de novo sempre que quiser recomençar.
5. Confira a última consulta do script: as contagens devem ser iguais às da tabela abaixo. Diferente do MySQL, o Postgres não tem um comando para "trocar de banco" dentro da mesma sessão; uma vez conectado a `pratica_sql`, todas as atividades já têm as tabelas disponíveis.

Tabela	Linhas
clientes	200
transacoes	1200
itens_transacao	800
passageiros	150

Tabela	Linhas
motoristas	40
corridas	3000

Dicionário de dados

A base tem dois cenários: um de **transações financeiras** e um de **corridas de aplicativo**. Os valores estão em reais.

Tabela	Colunas	Liga com	Observação
clientes	id, nome, email, segmento, data_cadastro	transacoes.cliente_id	segmento: Varejo, Premium, Corporate ou Startup
transacoes	id, cliente_id, data, valor, status	clientes.id	status: concluída, pendente, cancelada ou estornada; período de 01/01/2024 a 30/09/2025
itens_transacao	id, transacao_id, produto, quantidade, preco_unitario	transacoes.id	1 a 3 itens por transação, apenas nas transações 1 a 400
passageiros	id, nome, email, telefone, data_cadastro	corridas.passageiro_id	telefone no formato (21) 9XXXX-XXXX
motoristas	id, nome, data_cadastro, nota	corridas.motorista_id	nota vai de 3,5 a 4,9
corridas	id, passageiro_id, motorista_id, data_hora, valor	passageiros.id, motoristas.id	mesmo período das transações

Regras do jogo

- **Data de referência: 30/09/2025.** Sempre que a tarefa falar em "hoje" ou "últimos dias", use `DATE '2025-09-30'` no lugar de `CURRENT_DATE`. Assim todos chegam aos mesmos números.
- **A base tem sujeiras de propósito:** textos mal padronizados, valores ausentes e registros sem par. Descobri-las faz parte do exercício.
- Cada atividade tem três partes: **Ao vivo** (acompanhe e digite junto), **Tarefa de fixação** (resolva sozinho) e **Confira** (números que sua resposta deve reproduzir).
- Se o seu resultado não bater com o "Confira", releia o enunciado antes de mexer na consulta. Na maioria das vezes o erro está no filtro ou no tipo de JOIN.

Dia 1: Sumarização e tratamento de dados

No Dia 1 você transforma linhas soltas em indicadores usando uma tabela por vez: agregações, limpeza de texto, datas e categorias com `CASE`.

Atividade 1.1: Painel de KPIs por status

Objetivo: montar uma consulta de indicadores aos poucos e entender o que cada função de agregação realmente conta.

Ao vivo. Antes de rodar cada versão, tente prever quantas linhas vão voltar.

1. Faturamento por status:

```
SELECT status, SUM(valor) AS faturamento
FROM   transacoes
GROUP BY status;
```

2. Acrescente o número de operações e o ticket médio:

```
SELECT status,
       COUNT(*) AS ops,
       SUM(valor) AS faturamento,
       AVG(valor) AS ticket
FROM   transacoes
GROUP BY status;
```

3. Acrescente `COUNT(DISTINCT cliente_id) AS clientes`. Por que "clientes" é tão menor que "ops"?

4. Compare o que cada contagem enxerga:

```
SELECT COUNT(*) AS linhas, COUNT(valor) AS com_valor
FROM transacoes;
```

Os dois números são diferentes. O que isso diz sobre o `AVG(valor)` do passo 2?

Tarefa de fixação. Considerando apenas as transações de 2025, liste por `status` o total, o ticket médio e o número de clientes únicos. Ordene do maior total para o menor.

Dica: o período se filtra no `WHERE`, antes de agrupar. Um intervalo de datas se escreve com o literal `DATE '2025-01-01'`.

Confira: 4 linhas. A primeira é `concluida`, com total de 73.831,00, ticket médio de 245,29 e 115 clientes. A última é `estornada`, com total de 11.868,80 e 19 clientes.

Atividade 1.2: Higienização de cadastro

Objetivo: perceber como espaços e maiúsculas fazem o mesmo valor parecer diferente, e corrigir isso com funções de texto.

Ao vivo.

1. Conte os passageiros por domínio de e-mail, sem tratar nada:

```
SELECT SPLIT_PART(email, '@', 2) AS dominio,
       COUNT(*) AS passageiros
FROM passageiros
GROUP BY dominio
ORDER BY dominio;
```

Só existem 5 provedores de e-mail na base. Por que voltaram 14 linhas?

2. Investigue com colchetes e tamanhos:

```
SELECT id,
       '[' || email || ']' AS email,
       LENGTH(email) AS tamanho,
       LENGTH(TRIM(email)) AS tamanho_limpo
FROM passageiros
WHERE email <> TRIM(LOWER(email))
ORDER BY id
LIMIT 10;
```

3. Repita a contagem do passo 1 trocando `email` por `TRIM(LOWER(email))` dentro do `SPLIT_PART`. Agora devem sobrar 5 linhas.

Tarefa de fixação. Monte uma lista de contatos padronizada dos 10 primeiros passageiros (por `id`), com as colunas `nome`, `email_limpo`, `usuario` (a parte antes do @) e `tel_mascarado` (o telefone no formato `****1234`). Depois, descubra quantos passageiros têm e-mail que precisou de limpeza.

Dica: `RIGHT(telefone, 4)` pega os 4 últimos caracteres; `CONCAT` junta com os asteriscos (o Postgres tem `CONCAT`, igual ao MySQL). `SPLIT_PART(texto, '@', 1)` pega a parte antes do @.

Confira: a primeira linha é `Passageiro 001`, `passageiro1@gmail.com`, `passageiro1`, `****4729`. O total de e-mails sujos é 55.

Atividade 1.3: Faturamento mensal e faixas de valor

Objetivo: combinar função de data, `GROUP BY` e `CASE` para criar categorias e indicadores condicionais.

Ao vivo.

1. Faturamento por mês das corridas:

```
SELECT TO_CHAR(data_hora, 'YYYY-MM') AS mes,
       SUM(valor)                      AS faturamento
FROM   corridas
GROUP BY mes
ORDER BY mes;
```

2. Classifique cada corrida por faixa de valor:

```
SELECT valor,
       CASE WHEN valor > 100 THEN 'Premium'
            WHEN valor >= 40 THEN 'Média'
            ELSE 'Econômica' END AS faixa
FROM   corridas
LIMIT 10;
```

Depois agrupe por `faixa` e conte. Você deve encontrar 639 corridas Econômicas, 1.293 Médias e 1.068 Premium.

3. Junte as duas ideias com agregação condicional:

```
SELECT TO_CHAR(data_hora, 'YYYY-MM') AS mes,
       SUM(CASE WHEN valor > 100 THEN valor END) AS premium,
       SUM(CASE WHEN valor <= 100 THEN valor END) AS demais
FROM   corridas
GROUP  BY mes
ORDER  BY mes;
```

O `CASE` não tem `ELSE`. O que ele devolve quando nenhuma condição é verdadeira, e por que o `SUM` não se importa?

Tarefa de fixação. Calcule o tempo de casa dos motoristas em anos completos, usando a data de referência `DATE '2025-09-30'`. Classifique em `menos de 2 anos`, `2 a menos de 5 anos` e `5 anos ou mais`, e conte quantos motoristas há em cada faixa.

Dica: no Postgres, anos completos entre duas datas se calculam com `DATE_PART('year', AGE(data_mais_recente, data_mais_antiga))` — repare que a data mais recente vem primeiro dentro do `AGE`. Repita a expressão em cada `WHEN` e agrupe pelo apelido da faixa.

Confira: 7 motoristas com menos de 2 anos, 20 com 2 a menos de 5 anos e 13 com 5 anos ou mais (total 40).

Dia 2: Relacionamentos e filtros agregados

No Dia 2 você cruza tabelas com `JOIN` e aprende a filtrar grupos com `HAVING`, sem perder linhas nem inflar totais sem perceber.

Atividade 2.1: Enriquecer a transação com o cadastro

Objetivo: usar `INNER JOIN` para trazer dados do cliente e descobrir o que ele deixa de fora.

Ao vivo.

1. Faturamento por segmento:

```
SELECT c.segmento, SUM(t.valor) AS faturamento
FROM   transacoes t
INNER JOIN clientes c ON c.id = t.cliente_id
GROUP  BY c.segmento
ORDER  BY faturamento DESC;
```

2. Some manualmente as 4 linhas do resultado e compare com o total da tabela inteira:

```
SELECT SUM(valor) FROM transacoes;
```

O total da tabela é 285.876,82, mas os segmentos somam 281.028,82. Para onde foram os 4.848,00 de diferença? Guarde a pergunta: a atividade 2.2 responde.

Tarefa de fixação. Considerando apenas as transações **concluída**, calcule o ticket médio por segmento. Mostre só os segmentos com **mais de 170 transações concluídas** e ordene do maior ticket para o menor.

Dica: o **WHERE** escolhe o status; o **HAVING** filtra os grupos pela contagem.

Confira: 3 linhas: Varejo (ticket 249,78; 177 transações), Startup (238,45; 175) e Corporate (237,03; 183). O Premium, com 165, fica de fora.

Atividade 2.2: Auditoria de integridade com LEFT, RIGHT e FULL JOIN

Objetivo: usar os JOINS externos para encontrar registros sem par e fazer os totais fecharem.

Ao vivo.

1. Refaça o faturamento com **LEFT JOIN**, dando nome a quem não tem cadastro:

```
SELECT COALESCE(c.segmento, 'Sem cadastro') AS grupo,
       SUM(t.valor) AS faturamento
FROM   transacoes t
LEFT JOIN clientes c ON c.id = t.cliente_id
GROUP BY grupo
ORDER BY faturamento DESC;
```

Apareceu uma linha nova, **Sem cadastro**, com 4.848,00. A soma agora bate com a tabela inteira.

2. Liste essas transações órfãs (devem ser 20):

```
SELECT t.id, t.cliente_id, t.valor
FROM   transacoes t
LEFT JOIN clientes c ON c.id = t.cliente_id
WHERE  c.id IS NULL;
```

3. Agora o outro lado: clientes que nunca compraram (devem ser 10, ids 191 a 200):

```
SELECT c.id, c.nome
FROM   transacoes t
RIGHT JOIN clientes c ON c.id = t.cliente_id
WHERE  t.id IS NULL;
```

Reescreva a mesma consulta com `LEFT JOIN`, invertendo a ordem das tabelas. O resultado é idêntico.

4. O Postgres tem **FULL OUTER JOIN** nativo (o MySQL não tem, e por isso o simula com `UNION`). Rode direto:

```
SELECT t.id AS transacao_id, c.id AS cliente_id
FROM   transacoes t
FULL OUTER JOIN clientes c ON c.id = t.cliente_id;
```

Voltam 1.210 linhas: as 1.200 transações mais os 10 clientes sem compra.

5. Agora veja por que o truque do `UNION` (necessário no MySQL) é traiçoeiro, mesmo no Postgres. Rode as duas versões abaixo e compare o número de linhas.

Versão A:

```
SELECT c.nome, t.valor
FROM   transacoes t LEFT JOIN clientes c ON c.id = t.cliente_id
UNION
SELECT c.nome, t.valor
FROM   transacoes t RIGHT JOIN clientes c ON c.id = t.cliente_id;
```

Versão B:

```
SELECT t.id AS transacao_id, c.id AS cliente_id
FROM   transacoes t LEFT JOIN clientes c ON c.id = t.cliente_id
UNION ALL
SELECT t.id, c.id
FROM   transacoes t RIGHT JOIN clientes c ON c.id = t.cliente_id
WHERE  t.id IS NULL;
```

A versão A devolve 1.200 linhas e a B devolve 1.210, igual ao `FULL OUTER JOIN` nativo do passo 4. Onde a A perdeu 10 linhas? Pista: o `UNION` (sem `ALL`) remove linhas idênticas em qualquer banco, e o que muda entre A e B é quais colunas o `SELECT` escolhe.

Tarefa de fixação. Liste os clientes que não fizeram nenhuma compra nos 90 dias anteriores a `DATE '2025-09-30'`, ou seja, com compras apenas antes de 02/07/2025 ou nenhuma compra. Use `LEFT JOIN`. Escreva a condição de data primeiro no `WHERE` e depois no `ON`, e compare os dois resultados. Só um está certo.

Dica: `DATE '2025-09-30' - INTERVAL '90 days'` calcula o início da janela.

Confira: a versão correta devolve 58 clientes. Se a sua devolveu 0, a condição de data está no lugar errado: no `WHERE`, ela elimina justamente as linhas sem par que você queria encontrar.

Atividade 2.3: Duelo WHERE × HAVING

Objetivo: saber quando o filtro acontece (antes ou depois de agrupar) e perceber como um `JOIN` pode inflar uma soma.

Ao vivo. Cada consulta abaixo será projetada. **Antes de rodar, vote: roda, dá erro ou devolve o que se espera?** Depois execute e discuta.

Consulta A:

```
SELECT status, SUM(valor) AS total
FROM   transacoes
WHERE  SUM(valor) > 1000
GROUP BY status;
```

Consulta B:

```
SELECT status, SUM(valor) AS total
FROM   transacoes
GROUP BY status
HAVING valor > 0;
```

Consulta C:

```
SELECT status, SUM(valor) AS total
FROM   transacoes
WHERE  data >= DATE '2025-01-01'
GROUP BY status
HAVING SUM(valor) > 20000;
```

Consulta D (igual à C, mas com o apelido no `HAVING`):

```
SELECT status, SUM(valor) AS total
FROM   transacoes
WHERE  data >= DATE '2025-01-01'
GROUP BY status
HAVING total > 20000;
```

Resultados esperados no Postgres: A dá o erro `aggregate functions are not allowed in WHERE`, porque o `WHERE` roda antes de existir qualquer soma. B dá o erro `column "transacoes.valor" must appear in the GROUP BY clause or be used in an aggregate function`, porque depois de agrupar não existe mais um `valor` individual. C devolve `concluida` (73.831,00) e `pendente` (22.256,37). **D também dá erro no Postgres** (`column "total" does not exist`): diferente do MySQL, o Postgres não aceita apelido do `SELECT` dentro do `HAVING`. É uma boa demonstração de que escrever `SUM(valor)` por extenso no `HAVING`, como na consulta C, é mais portátil entre bancos.

Fixe a ordem em que o banco executa: `FROM/JOIN`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`.

Bônus: quando o JOIN infla a soma.

1. Some as transações 1 a 400 (só elas têm itens):

```
SELECT SUM(valor) FROM transacoes WHERE id <= 400;
```

Resultado: 95.646,94.

2. Some o mesmo campo depois de juntar com os itens:

```
SELECT SUM(t.valor)
FROM   transacoes t
INNER JOIN itens_transacao i ON i.transacao_id = t.id;
```

Resultado: 191.389,02, quase o dobro. Cada transação aparece uma vez por item (400 linhas viram 800), e o `SUM` soma todas as repetições. Um `JOIN` de 1 para muitos multiplica linhas. Quando a métrica mora no lado "muitos", some ela lá:

```
SELECT COUNT(DISTINCT t.id)           AS transacoes,
       SUM(i.quantidade * i.preco_unitario) AS receita_itens
FROM   transacoes t
INNER JOIN itens_transacao i ON i.transacao_id = t.id;
```

Resultado: 400 transações e 86.481,00 de receita de itens. Nesta base, o valor da transação e a soma dos seus itens não coincidem; o exemplo serve para mostrar a mecânica.

Tarefa de fixação. Escreva uma única consulta que use `JOIN`, `WHERE`, `GROUP BY`, `HAVING` e `ORDER BY` para responder: quais segmentos têm faturamento acima de R\$ 42.000, considerando apenas transações concluídas? Ordene do maior faturamento para o menor.

Dica: escreva na ordem em que o banco executa (veja a lista acima) e teste cada cláusula antes de acrescentar a próxima.

Confira: 2 linhas: Varejo (44.211,29) e Corporate (43.375,71). Startup (41.728,42) e Premium (41.487,08) ficam de fora.

Dia 3: Estruturação lógica e modularidade

No Dia 3 você organiza consultas complexas em etapas nomeadas e guarda o resultado para reutilizar, primeiro no banco (view) e depois no Python.

Atividade 3.1: Subconsultas como filtro

Objetivo: usar o resultado de uma consulta como valor de comparação dentro de outra.

Ao vivo.

1. Transações acima da média geral:

```
SELECT id, cliente_id, valor
FROM   transacoes
WHERE  valor > (SELECT AVG(valor) FROM transacoes);
```

A média é 243,09 e voltam 545 linhas. A subconsulta roda uma vez ou uma vez por linha? (Ela não depende da linha de fora, então roda uma única vez.)

2. Agora um nível a mais: clientes cujo gasto total está acima do gasto médio por cliente. É uma média de somas, então a subconsulta precisa agregar duas vezes:

```

SELECT cliente_id, SUM(valor) AS gasto
FROM   transacoes
GROUP BY cliente_id
HAVING SUM(valor) > (
    SELECT AVG(g.gasto)
    FROM   (SELECT SUM(valor) AS gasto
            FROM   transacoes
            GROUP BY cliente_id) g
);

```

O gasto médio por cliente é 1.496,74 e 89 clientes ficam acima dele. Note que a subconsulta interna aparece no `FROM`, como uma tabela temporária.

Tarefa de fixação. Liste os motoristas com número de corridas **acima da média de corridas por motorista**. Mostre o nome do motorista e a quantidade de corridas, da maior para a menor.

Dica: calcule primeiro, separadamente, quantas corridas cada motorista fez; depois use esse resultado como base da média. Para trazer o nome, junte com `motoristas`.

Confira: a média é 75 corridas por motorista e 11 motoristas ficam acima dela. Os três primeiros são Motorista 01 (298), Motorista 02 (163) e Motorista 03 (134).

Atividade 3.2: Da subconsulta aninhada para a CTE

Objetivo: reescrever uma consulta ilegível em etapas nomeadas com `WITH`.

Ao vivo.

1. A média do faturamento mensal com subconsulta aninhada. Tente ler em voz alta:

```

SELECT AVG(t) AS media
FROM   (SELECT TO_CHAR(data, 'YYYY-MM') AS m, SUM(valor) AS t
        FROM   transacoes
        GROUP BY m) x;

```

2. A mesma consulta com CTE:

```
WITH mensal AS (
  SELECT TO_CHAR(data, 'YYYY-MM') AS mes,
         SUM(valor)                AS total
  FROM   transacoes
  GROUP BY mes
)
SELECT AVG(total) AS media
FROM   mensal;
```

Os dois devolvem 13.613,18. A base tem 21 meses, de 2024-01 a 2025-09.

3. Encadeie duas CTEs: a segunda usa a primeira.

```
WITH mensal AS (
  SELECT TO_CHAR(data, 'YYYY-MM') AS mes,
         SUM(valor)                AS total
  FROM   transacoes
  GROUP BY mes
),
media AS (
  SELECT AVG(total) AS media_mensal
  FROM   mensal
)
SELECT mensal.mes, mensal.total, media.media_mensal
FROM   mensal CROSS JOIN media
ORDER BY mensal.mes;
```

Por que o `CROSS JOIN` é seguro aqui? Porque `media` tem uma única linha: cada mês só ganha a média ao lado, sem multiplicar nada.

Tarefa de fixação. Usando CTE, liste os meses cujo faturamento ficou **acima da média mensal**, mostrando `mes`, `total` e a `diferenca` em relação à média (arredondada em 2 casas). Ordene por mês.

Dica: reaproveite as duas CTEs do passo 3 e acrescente um `WHERE` na consulta final.

Confira: 10 dos 21 meses. O primeiro é 2024-03 (total 14.250,71; diferença 637,53) e a maior diferença é de 2024-12 (1.155,28).

Atividade 3.3: View para o dashboard e ponte para o Python

Objetivo: guardar uma consulta consolidada como view e consumir o resultado em um DataFrame.

Ao vivo.

1. Crie a view do faturamento por segmento (a sintaxe de `CREATE VIEW` é igual no Postgres e no MySQL):

```
CREATE VIEW fat_segmento AS
SELECT c.segmento, SUM(t.valor) AS tot
FROM   transacoes t
INNER JOIN clientes c ON c.id = t.cliente_id
GROUP BY c.segmento;
```

2. Consulte a view como se fosse uma tabela:

```
SELECT * FROM fat_segmento WHERE tot > 70000 ORDER BY tot DESC;
```

Voltam 3 segmentos: Corporate (71.106,22), Premium (70.601,00) e Startup (70.375,30). O Varejo (68.946,30) fica de fora.

3. Insira uma transação grande para um cliente do Varejo e repita o passo 2:

```
INSERT INTO transacoes (id, cliente_id, data, valor, status)
VALUES (1201, 4, DATE '2025-09-30', 5000.00, 'concluida');
```

Agora o Varejo aparece, com 73.946,30. A view não guarda cópia dos dados: ela executa a consulta de novo a cada uso.

4. **Desfaça o teste**, para os números continuarem batendo com os "Confira" das outras atividades:

```
DELETE FROM transacoes WHERE id = 1201;
```

A view deixou a consulta mais rápida? Não. Ela só dá nome a uma consulta; o custo de executá-la continua o mesmo.

Tarefa de fixação. Em duas etapas.

1. Crie a view `vw_kpi_mensal` com as colunas `mes` (formato `YYYY-MM`), `faturamento`, `operacoes` e `ticket_medio`, calculadas a partir de `transacoes`.
2. No Python, carregue a view em um DataFrame e plote o faturamento por mês. Instale antes as bibliotecas com `pip install pandas sqlalchemy psycopg2-binary matplotlib` (use `psycopg2-binary`, não `psycopg2`, para não precisar compilar nada).

```
import pandas as pd
from sqlalchemy import create_engine

engine =
create_engine("postgresql+psycopg2://USUARIO:SENHA@localhost/pratica_sql")
df = pd.read_sql("SELECT * FROM vw_kpi_mensal ORDER BY mes", engine)
df.plot(x="mes", y="faturamento", kind="line", marker="o")
```

Substitua `USUARIO` e `SENHA` pelas credenciais do seu Postgres.

Dica: a view é um `SELECT` com `TO_CHAR`, `SUM`, `COUNT(*)` e `AVG`, agrupado por mês.

Confira: a view tem 21 linhas. A primeira é 2024-01, com faturamento de 13.061,88, 58 operações e ticket médio de 233,25. No Python, `df["faturamento"].mean()` deve dar 13.613,18, o mesmo valor da atividade 3.2, e 10 meses ficam acima dessa média.

Checklist de autoavaliação

Marque o que você já faz sem consultar o material; o que ficar em branco é o que vale revisar antes da próxima aula.

Dia 1

- ☐ Explico a diferença entre `COUNT(*)`, `COUNT(coluna)` e `COUNT(DISTINCT coluna)`.
- ☐ Sei por que `AVG` ignora `NULL` e como isso muda o resultado.
- ☐ Padronizo texto com `TRIM` e `LOWER` antes de comparar ou agrupar.
- ☐ Crio categorias com `CASE` e faço agregação condicional.

Dia 2

- ☐ Escolho entre `INNER`, `LEFT` e `RIGHT JOIN` conforme a pergunta de negócio.
- ☐ Encontro registros sem par com `LEFT JOIN` e `IS NULL`.
- ☐ Sei por que a condição de período vai no `ON`, e não no `WHERE`, de um `LEFT JOIN`.
- ☐ Explico a diferença entre `WHERE` e `HAVING` e a ordem de execução das cláusulas.
- ☐ Reconheço quando um `JOIN` de 1 para muitos infla uma soma.

Dia 3

- ☐ Uso subconsulta no `WHERE` e no `FROM`.
- ☐ Transformo subconsultas aninhadas em CTEs.
- ☐ Sei o que uma view guarda (a consulta, não os dados) e o que ela não faz (acelerar).

- ☐ Carrego o resultado de uma view em um DataFrame.

Para treinar mais, adapte as tarefas ao outro cenário da base sempre que fizer sentido: por exemplo, repita a lógica de `transacoes` em `corridas`, ou a de `passageiros` em `clientes`.